Architectural Deep Dive

Neural Network Layer Computation & Forward Propagation



A technical reference covering neurons, layers, notation, and inference

---

Prepared as a structured technical summary · 7 sections

1. Fundamental Building Blocks: The Neuron and the Layer

The fundamental building block of most modern neural network architectures is the layer. As technical architects, we must conceptualise the transition from a single neuron—a unit implementing a discrete logistic regression—to a layer, which acts as a collective processing unit. While an individual neuron computes a single activation value, a layer inputs a vector of numbers and outputs a new vector of activations. This vectorised output is essential; it allows the subsequent layer to "see" and process multiple abstract features simultaneously.

We define our architecture based on the "demand prediction" use-case (e.g., predicting if a t-shirt will be a "top seller"). In this standard model:

- Layer 0 (Input Layer): Comprises four initial features, such as price and shipping cost.
- Layer 1 (Hidden Layer): Contains three neurons. Each neuron represents a higher-level concept — for instance, one neuron might compute "affordability" while another computes "brand awareness."
- Layer 2 (Output Layer): Consists of a single neuron that provides the final probability.

2. Mathematical Notation and Indexing Conventions

To navigate the complexity of deep architectures, we adhere to a rigorous notational framework. Precision in indexing is vital for the correct labelling and implementation of the data flow.

Standard Notational Conventions

Notation	Definition
Superscript [l]	Indexes the specific layer (e.g., $a^{[1]}$ is the activation vector from layer 1).
Subscript j	Indexes an individual unit/neuron within a layer.
$w_j^{[l]}$	The weight vector associated with the j-th unit in layer l.
$b_j^{[l]}$	The scalar bias parameter for the j-th unit in layer l.
$a^{[0]}$	The special designation for the input feature vector x.
g(z)	The activation function, specifically the sigmoid/logistic function.

Architectural Rule of Thumb

When naming a network, we do not count the input layer (Layer 0). A "three-layer network" refers to an architecture with two hidden layers and one output layer. We designate the input features as $a^{[0]}$ to ensure the mathematical formulas remain consistent across the entire forward propagation sequence.

3. Anatomy of a Single Unit Computation

Each individual neuron j in layer l performs a computation that transforms the input vector into a scalar activation, $a_j^{[l]}$. This value often represents a probability; for example, in our demand model, Layer 1 might produce activations like 0.3 (affordability), 0.7 (awareness), and 0.2 (quality).

The mathematical formula for the activation of unit j in layer l is:

$$a_j^{[l]} = g(w_j^{[l]} \cdot a^{[l-1]} + b_j^{[l]})$$

The components of this computation are as follows:

- Dot Product: The neuron calculates the dot product between its unique parameter vector $w_j^{[l]}$ and the entire activation vector $a^{[l-1]}$ from the previous layer. Note that a is often written with a small arrow or bolded to denote its status as a vector.
- Bias Addition: The scalar bias $b_j^{[l]}$ is added to the result of the dot product.
- Sigmoid Function: The intermediate linear result z is passed through the activation function g(z) = 1 / (1 + e^{-z}) to map the output to a range between 0 and 1.

Every unit within a layer possesses its own distinct parameters (w and b), allowing different neurons to recognise different patterns within the same input vector.

Reference Implementation

The single-unit computation above translates directly into a short, vectorised routine:

PYTHON

```python
import numpy as np

def sigmoid(z):
    # activation function g(z)
    return 1 / (1 + np.exp(-z))

def unit_activation(w_j, a_prev, b_j):
    z = np.dot(w_j, a_prev) + b_j # dot product + bias
    return sigmoid(z) # a_j^[l]
```

4. Vectorised Layer Transitions

In forward propagation, individual activations are grouped into a collective vector $a^{[l]}$. This vector represents the “knowledge” of the layer and serves as the raw input for every neuron in the next stage of the network.

This creates a strict dependency chain: the computation for layer l cannot occur until the output vector $a^{[l-1]}$ is fully resolved. Information flows from left to right, beginning at the input x and propagating

through the layers until the final prediction is reached. This systematic, sequential flow is the essence of forward propagation.

Layer-level Forward Pass

PYTHON

```python
def dense_layer(a_prev, W, b):
    # W: stacked weight vectors, one row per unit
    # returns a^[l], the full activation vector
    z = W @ a_prev + b
    return sigmoid(z)
```

5. Case Study: Handwritten Digit Recognition (Inference)

To illustrate inference at scale, consider the task of recognising handwritten digits (0 or 1). We process an 8x8 pixel image, which provides 64 input features. In this context, pixel intensities range from 0 (black) to 255 (bright white).

The architecture follows a three-step sequential flow:

1. Layer 1: 25 neurons compute the activation vector $a^{[1]}$ from the 64 features of x.
2. Layer 2: 15 neurons compute $a^{[2]}$ using the 25 activations from Layer 1.
3. Layer 3 (Output Layer): 1 neuron computes the final scalar activation $a^{[3]}$ using the 15 activations from Layer 2.

We observe a typical architectural trend here: the number of hidden units generally decreases as the data moves closer to the output layer, effectively "funnelling" 64 raw pixels into a single conceptual prediction.

moving to a target

Layer Sizes at a Glance

Layer	Units / Output
Layer 0 (Input)	64 pixel features (8x8 image)
Layer 1	25 neurons $\rightarrow a^{[1]}$
Layer 2	15 neurons $\rightarrow a^{[2]}$
Layer 3 (Output)	1 neuron $\rightarrow a^{[3]}$ (final scalar)

6. Final Output and Binary Classification

The final activation $a^{[L]}$ (where L is the final layer) is the output of the neural network's function, also denoted as f(x). This value represents the probability that the input belongs to the positive class (e.g., the digit is '1' or the product is a 'top seller').

To produce a concrete binary prediction $\hat{y}$, we apply a threshold (usually 0.5):

- If $a^{[L]} \geq 0.5$, predict $\hat{y} = 1$.
- If $a^{[L]} < 0.5$, predict $\hat{y} = 0$.

By defining the network output as f(x), we maintain continuity with the principles of linear and logistic regression while acknowledging the increased complexity of the underlying multi-layer transformations.

7. Technical Summary of Forward Propagation

Forward propagation is defined by three critical characteristics:

- Sequential Processing: The architecture operates as a chain of dependencies. Each layer's output vector is the mandatory input for the subsequent layer, ensuring information is transformed progressively.
- Activation Functions: The function g is the primary mechanism for introducing non-linearity. Without this, the entire network—no matter how many layers—would collapse into a simple linear transformation.
- Inference vs. Learning: Forward propagation is the process of making predictions (inference) based on fixed parameters. The process of adjusting these parameters to improve accuracy is known as "backward propagation," a separate and mathematically distinct phase of the learning cycle.

Takeaway: Forward propagation is nothing more than a disciplined, layer-by-layer application of "dot product → add bias → activate": each layer consumes the previous layer's activation vector $a^{[l-1]}$, produces its own $a^{[l]}$, and passes it forward until the final activation $a^{[L]}$ yields the network's prediction f(x).